UADE / DESARROLLO DE APLICACIONES II

M5 RENTAS

Informe técnico y estado del PR hacia develop

Cambios, evidencia, decisiones pendientes y preparación de integración

DRAFT ABIERTO - NO MERGEADO

Actualizado: 29 de octubre de 2026

PR #78 | chore(m5): consolidar integración y calidad antes de develop

BASE

develop @ 27e6f6d11181da870dc3dcb4c6edc14da5155100

RAMA CANDIDATA

integration/m5-final-develop-20261028

SHA validado por el run

2eb704dd7e3fcbad9ba1bfc907151ec0e7842b8f

github.com/DAI2-Modulo-de-rentas-deudas-y-tributos/Modulo-5/pull/78

Validación backend

Maven clean verify: 568/568 PASS; 0 fallos, errores u omitidos. JaCoCo gate PASS. La aceptación HTTP operativa posterior falló por SELF_APPROVAL_FORBIDDEN; detalle y acción en pág. 12.

Bloqueos que mantienen NO-GO

PIT global no verificable (SCRUM-807). Cuatro CVE Spring pendientes de decisión formal (SCRUM-793). Aceptación HTTP y revisiones de responsables pendientes.

CI del SHA 2eb704d

Frontend, detección, estructura, Terraform, Maven y JaCoCo: SUCCESS. El job backend falló después en la aceptación PDF/bulk: el mismo actor no puede solicitar y aprobar una configuración. No se debilitó esa protección.

Cómo leer las páginas siguientes

Las páginas 2 a 11 conservan el análisis del corte previo y sus referencias de publicación son históricas. Esta portada y la página 12 actualizan el resultado de CI, las correcciones de pruebas y los bloqueos. El commit documental posterior no altera el código probado en 2eb704d.

1. Alcance y forma de leer este informe

Este documento ordena tanto lo trabajado históricamente en el proyecto como el cierre técnico de octubre. **No es un changelog exhaustivo de los 52 commits locales**: esa lista debe reconciliarse con el diff exacto antes del PR. Cada dato se presenta con su grado de comprobación.

Etiqueta	Significado
Consigna / alcance	Requisitos del TPO y documento de alcance de M5, no pruebas de implementación.
Histórico PR/Jira	Cambios y tickets registrados en el otro chat y en GitHub; pueden ser de otra rama o momento.
Reporte local	Salida técnica suministrada por el equipo; no reproducida contra un SHA remoto en este entorno.
GitHub observado	Existencia/título de PRs al corte; que exista un PR no significa merge a develop.
Pendiente	Validación o autorización todavía no cumplida, incluso si hay código escrito.

Qué cubre M5

- Gestionar configuración de tributos y valores; calcular obligaciones y liquidaciones con trazabilidad.
- Registrar deudas y pagos, saldos, planes, beneficios, exenciones y recargos sin romper reglas económicas.
- Exponer APIs REST según roles y separar entradas de eventos del broker, con auditoría e idempotencia.
- Mantener pruebas unitarias, integración PostgreSQL, seguridad, cobertura y contratos de integración.

Regla de integración

Preparar un PR no equivale a autorizar un merge. Para fusionar deben quedar identificados el SHA candidato, los commits/archivos revisados, los resultados CI/PIT/SCA y los aprobadores. La entrega/ACK de Core o M9 sigue pendiente de validación externa.

FUENTE / ALCANCE: TPO DAII; Documento de Alcance Módulo 5; informes y PRs históricos.

2. Arquitectura funcional: qué y por qué

Componente	Qué hace / por qué existe	Cómo se aborda
Tributos y configuración	Administra conceptos, importes y vigencia para que no se facture con reglas ambiguas.	Estados de configuración y validaciones de transición; repositorios para persistencia.
Liquidaciones y deudas	Calcula montos, vencimientos y saldos; permite auditar el origen de cada deuda.	Servicios de dominio, histórico y migraciones Flyway en PostgreSQL.
Pagos e imputaciones	Evita dobles cobros, conserva capital e intereses y hace los saldos reproducibles.	Transacciones, claves de idempotencia, registros de movimiento y pruebas de concurrencia.
Planes de pago	Distribuye deudas en cuotas sin perder principal, numeración ni intereses.	Ledger y tests explícitos para cantidad, numeración e importes; restricciones de elegibilidad.
Reversiones	Corrige pagos manteniendo evidencia, sin borrar hechos contables.	Flujos de solicitar/aprobar/rechazar y auditoría, en vez de borrar pagos.
Recargos, beneficios y exenciones	Aplica reglas y excepciones respetando fechas, sujetos y condiciones.	Servicios separados, reglas de elegibilidad y tests de casos límite.

Patrones técnicos relevantes

• **Arquitectura por capas:** controladores/API, servicios de negocio y repositorios/JPA desacoplados.

• **Repository:** abstrae consultas PostgreSQL (por ejemplo pagos, deudas y conceptos).

• **Strategy/reglas de dominio:** facilita elegir comportamientos de cálculo según configuración, donde esté implementado.

• **Transactional Outbox:** permite persistir el hecho de negocio y preparar su publicación sin perder trazabilidad.

FUENTE / ALCANCE: Alcance de M5, narrativa de PRs y diagnóstico técnico; no es una auditoría de todos los métodos.

3. Integración con Core y otros módulos

El sistema municipal es modular. M5 necesita conocer contribuyentes y hechos externos, pero las integraciones asíncronas no deben depender de accesos directos a las bases de otros módulos.

Situación	Decisión / resultado	Estado de validación
Eventos entrantes	Consumer y registro de eventos; deduplicación e idempotencia para evitar efectos económicos repetidos.	Pruebas locales; no inferir producción.
Eventos salientes	Transactional Outbox, publisher RabbitMQ y manejo de confirmaciones y devoluciones.	PR #73 en borrador; broker local verificado, no ACK real de Core/M9.
Contribuyentes	Provienen de M1/Core mediante eventos/contratos; la API M5 consultada no ofrecía alta directa.	Limitación observada en entorno de demo; revisar integración real.
Reintentos y errores	Auditoría de fallas, idempotencia y mecanismos de reintento para no perder ni duplicar datos.	La prueba de entrega externa y DLQ debe verificarse antes de cerrar contrato.

Decisión de arquitectura: el broker y los consumers son el límite de integración; no se recomienda crear un POST REST público para simular eventos de Tránsito/Habilitaciones en producción. Las APIs de observación o reproceso son operaciones distintas y requieren control de acceso.

Lo que falta demostrar

- Contrato de mensajes acordado y versionado con Core/M9, con claves e identificadores estables.
- Entrega real, confirmación/ACK y fallas reproducibles de broker externo; revisión de ACL.
- Compatibilidad de migraciones y datos entre ambientes sin usar credenciales ni tablas compartidas de otro módulo.

4. Evolución histórica por grupos de PR

Los registros siguientes permiten explicar el avance en la exposición y el PDF. **Los PR históricos no equivalen a los 52 commits actuales**; la rama final debe compararse con develop antes de afirmar su contenido.

PR / período	Resumen para explicar qué se incorporó
#45 y #47	Integración backend inicial; luego beneficios e imputaciones. En reportes históricos se citan 261 tests en #45.
#49	Tareas SCRUM-97/126/152/472/618, con cobertura y tests de entonces (296 tests, JaCoCo 92,47% según chat).
#50	Tareas SCRUM-342/344/383/384/385/539/556/578/611/612/613; 302 tests y 93,04% líneas en ese corte.
#54 a #56	Contratos Core V4, deuda vencida y eventos de dominio (según resumen de PR/Jira).
#57 a #59	Manejo de errores, respuestas 401/403 y validaciones más robustas.
#60 a #62	Reglas de importes, idempotencia y bloqueo/estado de integración; revisar estado por PR.

PRs de integración recientes visibles en GitHub

PR	Alcance y cautela
#70	Persistencia de planes/UUID; PR en borrador en la revisión previa.
#71	Strategy y pruebas/seguridad E2E; PR en borrador en la revisión previa.
#73	Transporte RabbitMQ local con Outbox; no implica aceptación Core externa.
#75	Mejoras frontend selectivas: sesión, accesibilidad, URL seguras, respuestas obsoletas; reporta 384 tests y 85,90% líneas.
#77	Propuesta de entorno PROD desde main, con infraestructura y aprobaciones propias. No agregarla automáticamente al PR a develop.

FUENTE / ALCANCE: Resumen del PR y Jira; GitHub PRs #70, #71, #73, #75, #77, al 29/10/2026.

5. Qué se probó y cómo interpretar los resultados

Control	Evidencia disponible	Lectura correcta
JDK/Maven	Temurin 17.0.20.1; Maven Wrapper 3.9.16.	Compatible con POM y CI (reporte local).
`clean verify`	568 tests, 0 fallas, 0 errores, 0 omitidos; 3:23.	Suite PASS local en candidata reportada.
Testcontainers	PostgreSQL 17.11 efímero; Flyway hasta V31.	Sin modificar la base compartida `rentas`.
JaCoCo	91,80% líneas; 89,47% instrucciones; 66,70% ramas.	Gate local de líneas PASS. No reemplaza PIT.
PIT LateCharge	49 mutaciones: 41 KILLED, 6 SURVIVED, 2 NO_COVERAGE, 0 TO.	Sonda de una clase; no representa score global.
PIT PaymentPlan	Dos sondas: 11 KILLED, 1 NO_COVERAGE, 0 errores y 0 TO.	Bloqueo de memoria puntual no reproducido; falta PIT global.
Git diff	`git diff --check` pasó en los worktrees reportados.	No certifica revisión funcional ni revisión del SHA remoto.

Por qué reforzamos tests de cuotas y auditoría

Una mutación de `i++` a `i--` en `PaymentPlanService.grant` dejó al minion sin memoria en una sonda. Se agregaron aserciones para exigir cantidad y numeración de cuotas (1, 2, 3), conservación de capital/intereses y ausencia de efectos/auditoría en casos inválidos. Dos sondas posteriores eliminaron ese mutante sin errores. No quedó demostrada una causa única del OOM, y un test que pasa no valida el denominador global.

FUENTE / ALCANCE: Informes de ejecución local de 28/10; no reportes de CI remoto de esta candidata.

6. SCRUM-807 - Estado del gate de mutaciones

PIT introduce cambios controlados en el bytecode; si los tests detectan el cambio, el mutante es KILLED. Si sobrevive, puede faltar una aserción. Un timeout/error de minion exige diagnóstico, y `NO_COVERAGE` puede deberse a una selección de pruebas incorrecta.

Decisión técnica	Justificación
14 clases de producción	Se conserva el perímetro original del gate, no una selección simplificada de servicios.
Mutadores DEFAULTS	El POM no enumera 14 operadores; usa los predeterminados de PIT. No se debe confundir objetivos con mutadores.
Umbral 60%	No se altera para lograr una aprobación artificial.
Timeout 60 s	Evita dos falsos timeouts observados con 5 s en LateCharge; no explica todos los 41 históricos.
1 hilo en sonda	Reduce presión de RAM frente a `threads=4` de la candidata; no prueba estabilidad global.
Sin `targetTests`	Filtrar clases de prueba generó 83 mutantes sin cobertura en sondas parciales de PlanWorkflow/Reversal.

Hallazgos que NO deben maquillarse

• Una corrida histórica interrumpida registró 41 timeouts; el CSV parcial preserva sólo 37 identidades.

• Una sonda de una clase arrastró 474 clases de tests al pre-scan, con memoria libre local de sólo 0,47 GiB. Aún no hubo ejecución remota de prueba.

• La estimación de 60-100 minutos para PIT global tiene alta incertidumbre. Su score sigue **NO VERIFICABLE**.

Siguiente paso aprobado técnicamente, aún no ejecutado: sonda GitHub Actions de **PaymentPlanService** con Testcontainers, Java 17, `threads=1`, `timeoutConstant=60000`, telemetría y artefactos. Después, evaluación separada del PIT global.

7. SCRUM-793 - Dependencias y seguridad

Se informaron actualizaciones menores compatibles y una verificación de dependencias; la ausencia de una ruta explotable identificada en el código no elimina que una biblioteca siga dentro del rango afectado.

CVE / dependencia	Precondición revisada	Decisión requerida
CVE-2026-47890 Spring WebMVC 6.2.19	No se encontró SSE con fragmentos de vista.	Dependencia afectada. Spring LOW / OSV CRITICAL: corregir o aceptar formalmente.
CVE-2026-47884 Spring WebMVC 6.2.19	No se encontró XsltView ni handler de vistas catch-all.	Dependencia afectada. Spring MEDIUM / OSV CRITICAL: resolver discrepancia y aprobar.
CVE-2026-47834 Spring Data JPA 3.5.13	Sort HTTP por allowlist; consulta Outbox paginada fija.	Dependencia afectada. Evaluar actualización OSS compatible o aceptación.
CVE-2026-47860 Spring AMQP 3.2.12	Consumidor desactivado por defecto; no se vio descompresión.	Dependencia afectada; verificar broker/ACL antes de habilitar consumidor.

Responsables: Seguridad decide riesgo residual, DevOps evalúa exposición/ambientes y dueños de backend validan mitigaciones. No asumir que parches Enterprise-only están disponibles. El informe SCA local y su versión exacta del POM deben revisarse como una unidad.

FUENTE / ALCANCE: Diagnóstico SCRUM-793 informado; avisos oficiales de Spring CVE-2026-47890, 47884, 47834 y 47860.

8. Sonda GitHub Actions aislada: propuesta

La máquina local quedó bajo presión de memoria antes de ejecutar mutantes. Se propone usar un runner de GitHub Actions sin tocar el CI habitual, para conservar mediciones y reportes reproducibles.

Resguardo	Implementación propuesta
Rama única	`push` sólo en `codex/scrum-807-pit-actions`, con marcador `[run-pit-probe]` en el último commit.
Dispatch futuro	`workflow_dispatch` reservado para cuando la definición exista en rama predeterminada; SHA requerido.
Permisos mínimos	`contents: read`, checkout sin credenciales persistentes, sin secrets ni deploy.
Un mutante-perímetro	El script exige 14 clases en el POM origen y genera una sola clase mutable de prueba (`PaymentPlanService*`).
Gate preservado	Mutadores por defecto, sin `targetTests`, threshold 60, factor original, timeout 60 s.
Observabilidad	Java, CPU, memoria, disco, Docker, duración, SHA, logs e inventario HTML/XML/CSV.
Límites	120 minutos por job, 105 por paso PIT, mínimo 5 GiB libres al inicio; artefacto `always()` por 14 días.

Almacenamiento: los 14 GB nominales del runner público son una referencia, no una garantía de disponibilidad real. Dependencias Maven, imágenes de Testcontainers, compilación y reportes comparten el disco. El workflow aborta al inicio si hay menos de 5 GiB libres; debe examinarse la telemetría de la sonda antes del global.

Validación efectuada en esta preparación: YAML parseado, activación confinada a una rama, permiso `read`, subida `always()`, sintaxis Python validada y 4 tests de contrato del POM aprobados con fixture sintético. Falta revisar el POM real de la candidata y ejecutar el workflow en GitHub.

9. Revisión de Jira y hallazgos pendientes

La auditoría QA del 27/10 enumera tickets de riesgo y mejora por área. A continuación se describen como **hallazgos auditados**, no como tickets cerrados; debe revisarse su estado actual en Jira antes del PR.

Área	Tickets auditados y motivo de revisión
Backend / Santiago	SCRUM-700 autoaprobación; 788 lock de login; 789 fechas futuras; 790 retry/DLQ; 792 escaneo de vencimientos; 793 deps; 795 timing login; 796 trazas/eventId; 799 comodines de búsqueda; 804 fuga de entidades JPA.
Otros controles	SCRUM-740 firma HMAC; 729 backoff; 716 comentarios de auditoría/autorización.
DevOps	SCRUM-791 ruta CloudFront/ALB; 797 perfil TEST; 798 headers; 800 Actions pin SHA; 723 gitleaks.
Frontend / contratos	SCRUM-802 configuración mock; SCRUM-803 documentación; 794 integración de ramas; 801 CODEOWNERS.

Cambio de responsabilidad

La candidata apila cambios de backend, frontend, QA, docs y DevOps. Una cuenta Git autora no prueba autoría funcional: atribuir y solicitar revisión según propietario técnico real. La modificación previa de `.github/workflows/ci.yml` requiere aprobación DevOps; las migraciones económicas/PO y el rescate frontend requieren su revisión específica.

FUENTE / ALCANCE: Auditoria_Modulo5_2026-10-27.pdf y reportes de candidato; no lectura en vivo de estados Jira.

10. Camino de aprobación hasta develop

1. Congelar alcance. Obtener diff íntegro de la candidata, revisar 52 commits, PRs apilados y 4 cambios locales; no publicar por arrastre.

2. Hacer commits separados. SCRUM-807 tests; SCRUM-793 POM/SCA; experimentación Actions en rama aislada, cada uno con responsable.

3. Aprobar experimento. DevOps autoriza rama/evento push, runner, costo y retención; Seguridad aprueba datos de logs.

4. Fijar SHA revisado. Crear commit inmutable completo, registrar SHA/archivos exactos y comprobar que GitHub ve esos bytes.

5. Correr sonda real. Push autorizado con marcador; revisar resultado, memoria, Docker y reportes de PaymentPlanService.

6. Decidir PIT global. Solo después de sonda estable; workflow separado y aprobación de recursos/perímetro; score >=60%.

7. Cerrar Seguridad. Mitigar o aceptar formalmente cada CVE; adjuntar análisis de exposición y versión efectiva.

8. Validar candidata. Repetir CI, Maven, cobertura, SCA, contratos y revisión de responsables sobre el SHA final.

9. Abrir Draft PR. Contra develop, con PDF, revisión de commits, pruebas, evidencia y limitaciones explícitas.

10. Autorizar merge. Revisiones aprobadas, checks verdes, SHA inalterado y autorización final. Registrar SHA de merge.

Estado actual de esa secuencia

La preparación documental y la sintaxis del workflow se pueden dar por terminadas. **Los pasos 1-10 que dependen del worktree local, permisos de GitHub, DevOps/Seguridad y una sonda real no están cerrados**. No se ejecutó PIT global, no hay PR nuevo ni merge a develop en esta intervención.

12. Estado del PR, CI y decisiones

Corte: 29/10/2026 | SHA analizado: 2eb704dd | Draft #78

Git / PR	PR #78 OPEN, DRAFT, MERGEABLE. integration/m5-final-develop-20261028 hacia develop. HEAD de run: 2eb704dd7e3fcbad9ba1bfc907151ec0e7842b8f. Sin merge ni deploy.
Checks	Frontend, application-detection, repository-structure y terraform: SUCCESS. Maven y JaCoCo: SUCCESS. Aceptación PDF/bulk, backend agregado y ci-required: FAILURE por el 403 funcional indicado abajo.
Maven / PostgreSQL	Temurin 17.0.21 en GitHub. `clean verify`: 568 tests, 0 fallos, 0 errores, 0 omitidos. PostgreSQL/Testcontainers activos; JaCoCo declaró cumplidos sus checks. El job continuó con aceptación operativa tras Maven.
Fallo de aceptación	POST /tax-configurations/1/approve devolvió 403 SELF_APPROVAL_FORBIDDEN. El runner creó, envió y aprobó la configuración con una misma sesión. La política de no autoaprobar fue aplicada correctamente.
Acción / ownership	Runner `scripts/test/aws_test_acceptance.py` creado históricamente por pNico1. Revisión solicitada en PR #78. Ajustar el escenario para crear/enviar con RENTAS y aprobar con otro SUPERVISOR. No cambiar la regla productiva ni el script ajeno en esta ronda.
Render PDF CI	Render y upload de evidencia pasaron en este run; Poppler se instaló tras Maven exitoso. El run anterior falló en render en cascada cuando Maven falló y omitió la instalación.
SCRUM-807 / PIT	NO VERIFICABLE. Sonda PaymentPlanService histórica: 11 KILLED, 1 NO_COVERAGE, sin MEMORY_ERROR/TIMED_OUT en dos ejecuciones. Falta PIT global de 14 clases, score >=60 y reportes HTML/XML/CSV completos.
SCRUM-793 / SCA	BLOQUEADO por decisión de Seguridad. Cuatro CVE Spring permanecen en rangos afectados; no se declaran corregidos ni aceptados.
Base / Core	develop @ 27e6f6d11181da870dc3dcb4c6edc14da5155100. RabbitMQ local no prueba entrega a Core/M9 ni consumer ACK; no se conectó sistema externo.

Decisión

GO para revisión en Draft; NO-GO para merge. Pendiente: corregir el runner de aceptación con aprobación de su responsable, lograr CI completo verde; obtener PIT global válido >=60%; decisión formal de Seguridad para los cuatro CVE y revisiones de DevOps, PO y responsables afectados.

PR: github.com/DAI2-Modulo-de-rentas-deudas-y-tributos/Modulo-5/pull/78